↳ reduces sum(x) and sum(x²)

LayerNorm removes both the mean and magnitude

RMSNorm removes only magnitude

↳ reduces only sum(x²)

→ for one token/row $x \in \mathbb{R}^D$, normalization happens across its dimension - not across batch

- LayerNorm -

$$\mu = \frac{1}{D}\sum_i x_i \qquad \sigma^2 = \frac{1}{D}\sum_i (x_i - \mu)^2$$

$$y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i$$

- centers values around zero
- scales them to roughly unit variance
- usually has learned 'gamma' and 'beta'

- RMSNorm -

$$RMS(x) = \sqrt{\frac{1}{D}\sum_i x_i^2 + \epsilon} \qquad y_i = \gamma_i \frac{x_i}{RMS(x)}$$

- does not subtract the mean
- controls only the vector's magnitude
- usually has 'gamma' and 'beta'
- needs less reduction work

the learned gamma lets the network restore useful feat-spec. scales

EXAMPLE: $x = [1, 2, 3]$

Layer => $[-1.225, 0, 1.225]$

RMS => [0.463, 0.926, 1.385]

- large activations can produce unstable gradients;
- tiny activations can make signals disappear;
- one unusually large feature can dominate a layer;
- components such as attention become sensitive to arbitrary input scale;
- optimization becomes poorly conditioned and harder to train.

LayerNorm and RMSNorm repeatedly bring each token's representation back to a predictable scale.

↳ stabilize internal representations so a deep network can train reliably

Why might scale normalization be enough?

Inside a Transformer, the token vector is not an ordinary measurement where zero has a fixed physical meaning. It is a learned representation produced by linear layers, attention, residual connections, and nonlinearities.

The network can often learn to manage feature offsets itself. The more important instability is usually the representation's magnitude growing or shrinking across many residual blocks.

RMSNorm addresses that directly:

$$\frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}}$$

It retains more of the original vector while fixing its scale.

Why LLMs often use RMSNorm

- Simpler statistic: only $\sum x_i^2$
- No mean subtraction
- Usually no learned bias
- Slightly fewer operations and parameters
- Works well with pre-norm residual architectures
- Often provides similar training stability to LayerNorm

It is not universally "better." LayerNorm offers stronger invariance and is still useful in many architectures. RMSNorm is a good architectural tradeoff when recentering does not provide enough benefit to justify the additional work.

! Remember

Grid
└ Blocks
└ Threads
grouped auto. into warps of 32

C++

```
kernel<<<100, 256>>>();
```

This means:

- `100` blocks
- `256` threads per block
- `100 × 256 = 25,600` total threads
- `256 / 32 = 8` warps per block

$$warps\_per\_block = \frac{threads\_per\_block}{32}$$

$$total\ threads = blocks \times threads\_per\_block$$

-> Threads can directly synchronize and share memory only with threads in the same block

- `100 × 8 = 800` total warps

Within one block:

Plain text

```
warp 0 = threads   0–31
warp 1 = threads  32–63
warp 2 = threads  64–95
...
warp 7 = threads 224–255
```

=> warp/block optimized_impl

C++

```
float block_sum =
    lane < warp_count
    ? warp_sums[lane]
    : 0.0f;
```

With 256 threads:

Plain text

```
warp_count = 8

first warp:
lane 0 loads warp_sums[0]
lane 1 loads warp_sums[1]
...
lane 7 loads warp_sums[7]
lanes 8–31 load 0
```

Direct comparison

Part	RMSNorm	LayerNorm
First statistic	$\sum x^2$	$\sum x$
Additional statistic	None	$\sum (x-\mu)^2$
Centers values	No	Yes
Scaling	`gamma`	`gamma`
Bias	Usually none	Usually `beta`
Clear CUDA input reads	2	3
Optimized CUDA input reads	2	2
Numerically robust reduction	FP32 sum of squares	Welford

"which intermediate values can remain in registers?"

=> registers are private to each thread

`float local_sum_sqr = 0.0f`

stays in register

local_sum_sqr += val * val

both val and accumulator stays in register

"How much memory traffic does fusion remove?"

=> fusion removes

2 x numel() x element size

for fp32 that is:

2 x 4 = 8 bytes per element

For a 4096x4096 fp32 tensor:

$4096^2 \times 8 = 128$ MiB

of temporary global-memory traffic is removed.

"Why is variance calculation numerically sensitive?"

=> two large, nearly equal numbers are subtracted

example: values = 100,00 $E[x^2] = 100,000,001$ "catastrophic cancellation"

variance = 1 $E[x]^2 = 100,000,000$

variance = 1

Accuracy hierarchy

From least to most robust:

1. `E[x²] - E[x]²`
2. Compute mean, then reduce `(x - mean)²`
3. Welford's online/parallel variance algorithm

The two-pass method is more stable:

```
C++

mean = sum(x) / N;
variance = sum((x - mean)²) / N;
```

Welford is normally preferred when stronger numerical robustness is required.

RMSNorm avoids the dangerous subtraction because it only calculates:

$$E[x^2]$$

"When does register pressure reduce performance ?"

=> every SM has a finite register file

If register demand becomes too high
the compiler spills values into local memory

↳ backed by device cache
much slower than registers

? "How should behavior change for large hidden dimensions"

=> A single warp is good for short rows but only have 32 workers

Small hidden dimensions

For something like $D \leq 128$:

```
Plain text

one warp → one row
multiple warps/rows per block
```

Benefits:

- no block-wide reduction;
- no inter-warp shared memory;
- several rows processed by one block;
- low synchronization overhead.

Medium hidden dimensions

For dimensions such as `512–8192`:

```
Plain text

one block → one row
256 threads → strided elements
warp reductions → shared warp totals
```

↳ this is used
- retains enough row parallelism while keeping the block-size within hardware limits

Very large hidden dimensions

Eventually, several problems appear:

- one block cannot have more than 1024 threads;
- each thread processes many elements;
- retaining the entire row creates excessive register pressure;
- long reductions accumulate more floating-point error;
- a giant Triton power-of-two tile may spill or fail to compile efficiently.